

SOFTWARE CHANGE IMPLEMENTATION REPORT

CR-001: Client-Side Search and Filtering of the Admin

Student List

Field	Details
Change Request ID	CR-004
Change Request Title	Student Input Validation and Strong Password Enforcement
Student Name	A.Paheershana
Student ID	JF/ICT/24/04
Implementation Status	Completed
Date	8 October 2026

1. Change Request Summary

1.1 Business Requirement

Administrators previously could add and edit student records without comprehensive validation of all student information. The change was required to improve student data quality, consistency, and account security.

The system must validate registration number, first name, last name, email address, phone number, and password information before student records are created or updated.

The system must also enforce a strong password policy when creating a new student account.

1.2 Existing Behaviour

Before implementation:

- The administrator could add and edit student records.
- Basic required-field validation was available.
- First and last names were not comprehensively validated.
- First and last names were not required to have a minimum of three characters.
- Invalid characters in names were not comprehensively rejected.
- Email format validation was not comprehensively enforced.
- Phone numbers were not strictly restricted to exactly 10 numeric digits.
- Strong password requirements were not comprehensively enforced.
- Server-side validation did not provide complete protection against invalid input submitted directly to the API.
- Existing Add, Edit, Delete, authentication, and authorization functionality was available.

1.3 Requested Change

A comprehensive validation facility was required for the administrator's student Add and Edit operations.

The validation must:

- Require a registration number.
- Require first and last names.
- Require a minimum of 3 characters for first and last names.
- Allow valid alphabetic characters for first and last names.
- Validate the email address format.
- Require a phone number containing exactly 10 digits.
- Reject phone numbers containing non-numeric characters.
- Reject phone numbers with fewer than 10 digits.
- Reject phone numbers with more than 10 digits.
- Require a password when creating a new student.
- Require passwords to contain at least 8 characters.
- Require at least one uppercase letter.
- Require at least one lowercase letter.
- Require at least one number.
- Require at least one special character.
- Provide clear validation error messages.
- Perform validation on both frontend and backend.
- Preserve existing Add and Edit functionality.

- Preserve existing Delete functionality.
- Preserve existing authentication and authorization.
- Continue storing passwords securely using hashing.
- Prevent invalid student information from being stored in the database.

2. Implementation

2.1 Implementation Summary

The CR-004 requirements were implemented in the administrator student management functionality.

Client-side validation was added to `StudentFormModal.jsx` to provide immediate feedback and prevent invalid form submission.

Server-side validation was added to `studentController.js` to ensure that invalid data cannot bypass frontend validation through direct API requests.

The implementation validates student registration number, first name, last name, email address, phone number, and password according to the defined requirements.

Strong password validation is applied when creating a new student. Existing password hashing using `bcryptjs` is preserved.

The implementation also preserves the existing Add, Edit, Delete, authentication, and authorization functionality.

No database schema changes were required.

2.2 Affected Files

File	Change	Purpose
<code>frontend/src/components/StudentFormModal.jsx</code>	Modified	Implemented client-side validation for student information and strong password requirements.
<code>backend/controllers/studentController.js</code>	Modified	Implemented reusable server-side validation for student Add and Edit operations.
<code>backend/models/studentModel.js</code>	Unchanged	Existing database operations were preserved.
Database	Unchanged	No database schema changes were required.

2.3 Technical Changes

Registration Number Validation

A registration number is required when adding or editing a student.

The validation:

- Checks that the registration number is not empty.
- Removes unnecessary leading and trailing spaces.
- Prevents submission when the registration number is missing.
- Existing duplicate registration number checking is preserved.

Name Validation

First name and last name are validated to:

- Be required.
- Contain at least 3 characters.
- Contain valid alphabetic characters and spaces.

Invalid values containing numbers or unsupported characters are rejected.

Email Validation

The email field:

- Is required.
- Must follow a valid email format.
- Is validated on both frontend and backend.
- Existing duplicate email checking remains available.

Phone Number Validation

The phone number must:

- Be required.
- Contain numeric characters only.
- Contain exactly 10 digits.

Phone numbers containing fewer than 10 digits or more than 10 digits are rejected.

The frontend also restricts phone input to numeric characters and limits the input to 10 digits.

Strong Password Validation

For new student accounts, the password must:

- Contain at least 8 characters.
- Contain at least one uppercase letter.
- Contain at least one lowercase letter.
- Contain at least one number.
- Contain at least one special character.

Example valid test password:

Student@123

Passwords continue to be securely hashed using bcryptjs before being stored.

Frontend Validation

Frontend validation was implemented in:

frontend/src/components/StudentFormModal.jsx

The frontend validates the information before sending the request to the backend.

This provides immediate feedback to the administrator and prevents unnecessary API requests for invalid input.

Backend Validation

Server-side validation was implemented in:

backend/controllers/studentController.js

The backend performs validation even when a request is sent directly to the API. This prevents users from bypassing the frontend validation.

2.4 Implementation Logic

- Administrator opens the Add or Edit Student form.
- Administrator enters student information.
- Frontend validation checks the entered values.
- If validation fails, a clear error message is displayed.
- If validation succeeds, the API request is sent.
- `studentController.js` receives the request.
- Server-side validation checks the submitted data again.
- If server-side validation fails, the request is rejected.
- Duplicate registration number and email checks are performed where applicable.
- For a new student, the password is securely hashed using `bcryptjs`.
- Valid information is passed to the existing student model.
- The existing database operation is performed.
- The administrator receives the appropriate success response.
- Existing Edit and Delete operations continue to use the correct `student_id`.

2.5 Error and Data Handling

The existing student-loading and API error handling remains unchanged.

The implementation provides clear validation messages for invalid student information.

Examples include:

- Registration number is required.
- First name is required.
- First name must be at least 3 characters.
- First name can contain only letters and spaces.
- Last name is required.
- Last name must be at least 3 characters.
- Last name can contain only letters and spaces.
- Please enter a valid email address.
- Phone number is required.
- Phone number must be exactly 10 digits.
- Phone number must contain only digits.
- Password is required.
- Password must be at least 8 characters.
- Password must contain at least one uppercase letter.
- Password must contain at least one lowercase letter.

- Password must contain at least one number.
 - Password must contain at least one special character.
- Invalid input is rejected before it can be stored in the database.

3. Testing & Verification

3.1 Testing Approach

The implementation was verified through frontend, backend, API, security, and regression testing.

Testing Types:

- Functional testing
- Positive testing
- Negative testing
- Boundary testing
- Validation testing
- Frontend testing
- Backend testing
- API testing
- Security testing
- UI testing
- Regression testing
- CRUD verification
- Authentication testing
- Authorization testing
- Error handling testing

Environment: Development

Browser: [Chrome / Edge / Firefox]

Test Role: Administrator

3.2 Test Cases

Test ID	Test Case	Expected Result	Actual Result	Status
TC-01	Submit form with empty required fields	Required-field validation messages are displayed	Validation messages displayed	Pass
TC-02	Enter first name with fewer than 3 characters	First name is rejected	First name rejected	Pass
TC-03	Enter last name with fewer than 3 characters	Last name is rejected	Last name rejected	Pass
TC-04	Enter invalid characters in first/last name	Invalid name is rejected	Invalid name rejected	Pass
TC-05	Enter an invalid email address	Email validation error is displayed	Email rejected	Pass
TC-06	Enter a phone number with fewer than 10 digits	Phone number is rejected	Phone number rejected	Pass
TC-07	Enter a phone number with more than 10 digits	Phone number is rejected	Phone number rejected	Pass
TC-08	Enter a non-numeric phone number	Non-numeric phone input is rejected	Invalid input rejected	Pass
TC-09	Enter a valid 10-digit phone number	Phone number is accepted	Valid phone accepted	Pass
TC-10	Enter a password shorter than 8 characters	Password is rejected	Password rejected	Pass

TC-11	Enter password without uppercase letter	Password is rejected	Password rejected	Pass
TC-12	Enter password without lowercase letter	Password is rejected	Password rejected	Pass
TC-13	Enter password without a number	Password is rejected	Password rejected	Pass
TC-14	Enter password without a special character	Password is rejected	Password rejected	Pass
TC-15	Enter a valid strong password	Password is accepted	Strong password accepted	Pass
TC-16	Create student using valid information	Student is successfully created	Student created successfully	Pass
TC-17	Create student using invalid information	Student creation is rejected	Invalid request rejected	Pass
TC-18	Edit student using valid information	Student is successfully updated	Student updated successfully	Pass
TC-19	Edit student using invalid information	Student update is rejected	Invalid update rejected	Pass
TC-20	Send invalid data directly to API	Backend validation rejects request	Request rejected	Pass
TC-21	Verify existing Delete functionality	Correct student is deleted	Delete worked correctly	Pass
TC-22	Verify administrator authentication and authorization	Existing security controls continue to work	Security controls worked	Pass

TC-23	Verify password hashing	Password is not stored as plain text	Password hashing retained	Pass
-------	-------------------------	--------------------------------------	---------------------------	------

3.3 Test Summary

Test Result	Count
-------------	-------

Total Test Cases	23
------------------	----

Passed	23
--------	----

Failed	0
--------	---

Pass Rate	100%
-----------	------

Overall Test Result: Pass

4. Acceptance Criteria Verification

ID	Acceptance Criterion	Verification	Status
AC-001	Registration number is required.	Required-field test	Met
AC-002	First name must contain at least 3 characters.	Name validation test	Met
AC-003	Last name must contain at least 3 characters.	Name validation test	Met
AC-004	First and last names accept valid alphabetic characters.	Name validation test	Met
AC-005	Invalid email format is rejected.	Email validation test	Met
AC-006	Phone number must contain exactly 10 digits.	Phone boundary test	Met
AC-007	Non-numeric phone numbers are rejected.	Phone validation test	Met

AC-008	Password must contain at least 8 characters.	Password validation test	Met
AC-009	Password must contain uppercase and lowercase letters.	Password validation test	Met
AC-010	Password must contain a number.	Password validation test	Met
AC-011	Password must contain a special character.	Password validation test	Met
AC-012	Valid student information can be successfully added.	Student creation test	Met
AC-013	Invalid student information cannot be added.	Negative testing	Met
AC-014	Valid student information can be successfully edited.	Student update test	Met
AC-015	Invalid student information cannot be updated.	Negative testing	Met
AC-016	Server-side validation prevents API validation bypass.	Direct API test	Met
AC-017	Passwords remain securely hashed.	Security verification	Met
AC-018	Existing Edit and Delete functionality continues to work.	Regression testing	Met
AC-019	Existing authentication and authorization continue to work.	Security regression testing	Met

5. Evidence

Five relevant screenshots should be attached to demonstrate the implementation and verification of CR-004.

Evidence 01 – Required Field and Name Validation

Related: AC-001, AC-002, AC-003, AC-004

Evidence:

The screenshot shows the 'Student Management System' Administrator Dashboard. A modal titled 'Add New Student' is open, prompting the user to 'Create a new student account.' The modal contains several input fields: 'Registration Number', 'First Name', 'Last Name', 'Email', 'Phone Number', and 'Password'. The 'Registration Number' field contains a single character 'I'. A red error message box is displayed over this field, stating 'Please fill out this field.' The background shows a table of existing students with columns for 'REGISTRATION NO.', 'NAME', and 'ACTIONS'.

This screenshot shows the same 'Add New Student' modal. The 'First Name' field now contains the character 'n'. A red error message box is displayed over this field, stating 'Please lengthen this text to 3 characters or more (you are currently using 1 character).' The 'Registration Number' field now contains the character 'n'. The background table and dashboard elements remain the same.

Figure 1: Administrator student form showing required field and name validation.

Evidence 02 – Email and Phone Number Validation

Related: AC-005, AC-006, AC-007

Evidence:

The figure consists of two screenshots of the 'Add New Student' form in the Student Management System. The first screenshot shows the form with the following data: Registration Number 'n', First Name 'raj', Last Name 'sami', Email 'admin@', and Phone Number (empty). A red error message is displayed: 'Please include an '@' in the email address. 'admin' is missing an '@'. The second screenshot shows the form with the following data: Registration Number 'REG003', First Name 'kumar', Last Name 'hera', Email 'kumar@gmail.com', and Phone Number '21'. A red error message is displayed: 'Phone number must contain exactly 10 digits.' Both screenshots show 'Cancel' and 'Create Student' buttons at the bottom of the form.

Student Management System
Administrator Dashboard

Students
Manage student accounts and profiles.

Add New Student
Create a new student account.

Registration Number: n, First Name: raj, Last Name: sami, Email: admin@, Phone Number: [empty]

Please include an '@' in the email address. 'admin' is missing an '@'.

Add New Student
Create a new student account.

Phone number must contain exactly 10 digits.

Registration Number: REG003, First Name: kumar, Last Name: hera, Email: kumar@gmail.com, Phone Number: 21

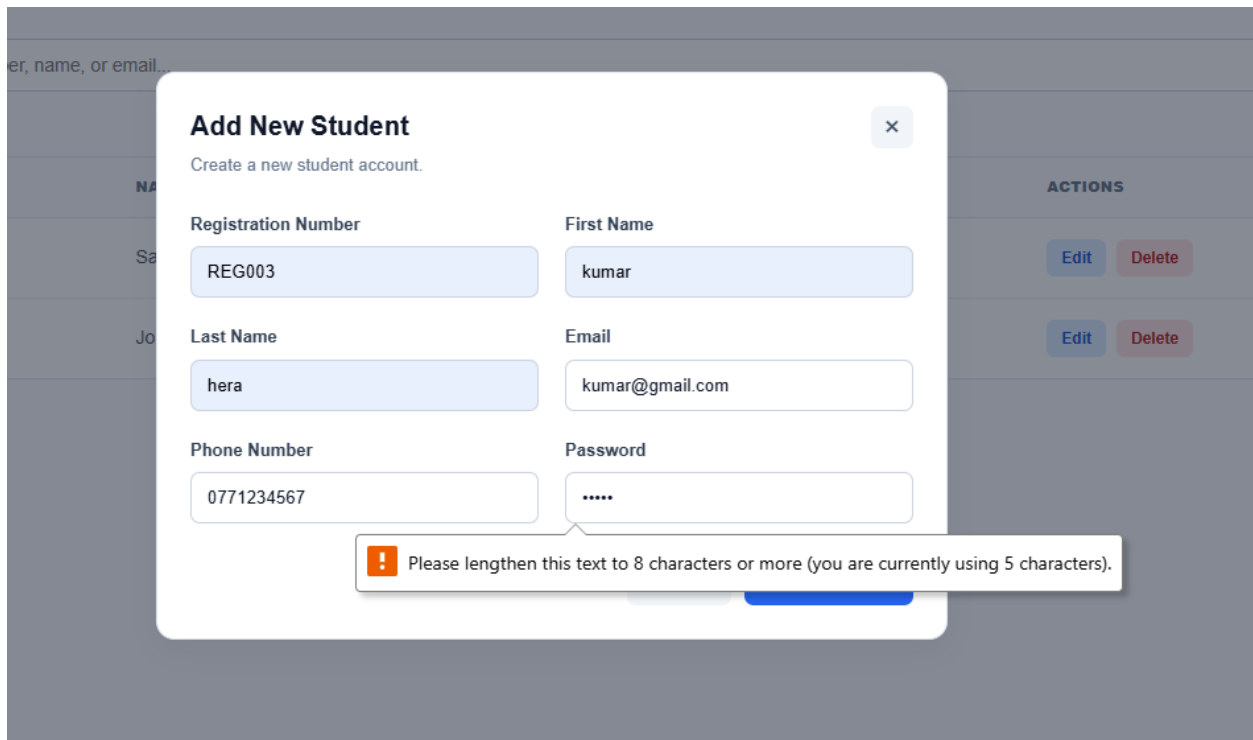
Cancel, Create Student

Figure 2: Email and phone number validation showing invalid input being rejected.

Evidence 03 – Strong Password Validation

Related: AC-008, AC-009, AC-010, AC-011

Evidence:



The screenshot displays a web application interface with a modal window titled "Add New Student". The modal contains a form with the following fields and values:

Registration Number	First Name
REG003	kumar
Last Name	Email
hera	kumar@gmail.com
Phone Number	Password
0771234567	

A validation error message is displayed below the Password field: "Please lengthen this text to 8 characters or more (you are currently using 5 characters)." The background shows a table with columns for "NAME" and "ACTIONS", with buttons for "Edit" and "Delete" visible.

Figure 3: Strong password validation requiring minimum length, uppercase, lowercase, number, and special character.

Evidence 04 – Successful Student Add/Edit

Related: AC-012, AC-014

Evidence:

[INSERT SCREENSHOT OF SUCCESSFULLY CREATED OR UPDATED STUDENT RECORD]

Figure 4: Successful student Add/Edit operation after valid input validation.

Manage student accounts and profiles

Manage student accounts and profiles

Search by registration number, name, or email...

3 students

Registration No.	Name	Email	Phone	Actions
REG004	kumar hera	kumar@gmail.com	0771234567	<button>Edit</button> <button>Delete</button>
REG002	Sarah Fernando	sarah@example.com	0723456789	<button>Edit</button> <button>Delete</button>
REG001	John Perera	john@example.com	0712345678	<button>Edit</button> <button>Delete</button>

Related: AC-013, AC-015, AC-016, AC-017, AC-018, AC-019

Evidence:

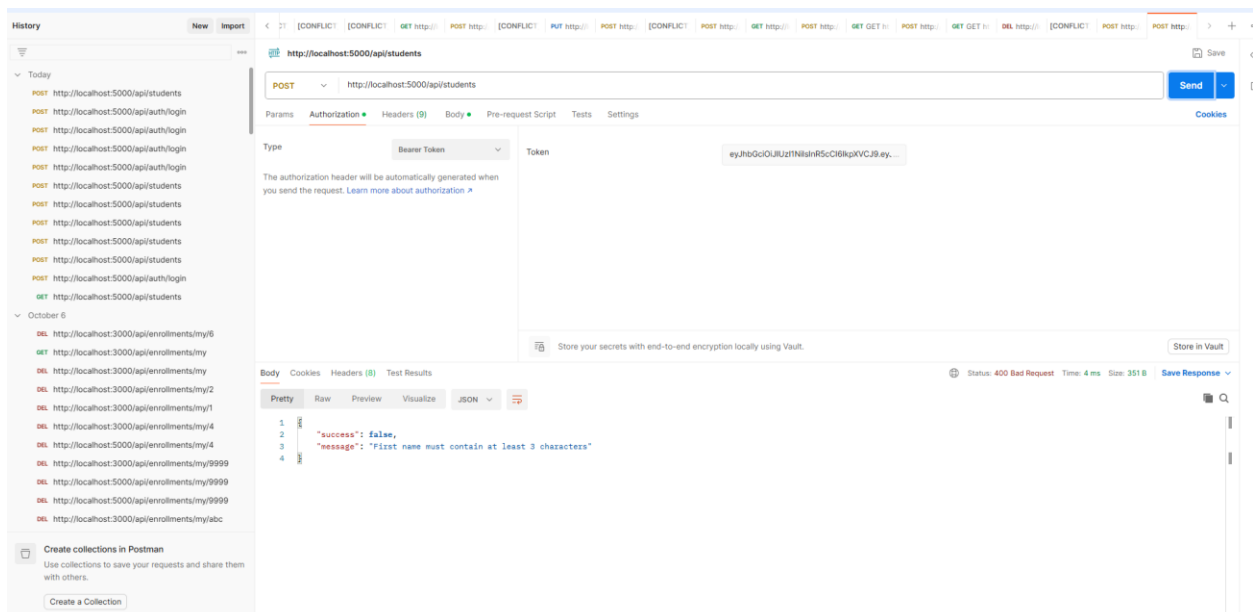


Figure 5: Server-side validation and preservation of existing student CRUD and security functionality.

6. Final Result

Item	Result
Requirements	Met
Implementation	Completed
Testing	Passed
Acceptance Criteria	All Met
Existing CRUD	Verified
Authentication / Authorization	Verified
Password Security	Verified
Final Status	Accepted

6.1 Final Summary

CR-004 was implemented as a comprehensive student input validation and strong password enforcement enhancement for the administrator student management functionality.

The implementation validates registration number, first name, last name, email address, phone number, and password information.

First and last names must contain at least three characters and valid alphabetic characters. Email addresses are checked against a valid format, while phone numbers are restricted to exactly ten numeric digits.

For new student accounts, passwords must contain at least eight characters, including uppercase and lowercase letters, a number, and a special character.

Validation is implemented at both frontend and backend levels. The frontend provides immediate feedback and prevents invalid form submission, while the backend prevents validation bypass through direct API requests.

The existing password hashing mechanism using bcryptjs has been preserved, ensuring that passwords are not stored as plain text.

No database schema changes were required. Existing Add, Edit, Delete, authentication, and authorization functionality was preserved.

The implemented functionality was tested using functional, negative, boundary, API, security, and regression testing. All defined test cases passed and all acceptance criteria were met.

6.2 Issues / Limitations

No significant issues identified.

The implemented validation currently accepts English alphabetic characters and spaces for first and last names according to the defined validation rules.

Student Name: _____A.Paheershana_____

Student ID: _____JF/ICT/24/04_____

Date: 8 October 2026

Signature: _____A.Paheershana_____

~ END OF REPORT ~

